

Actividad de evaluación (ABP)

Gestión de Bitácoras en Archivos de Texto (PHP)

Competencia a evaluar

El estudiante manipula archivos de texto en PHP para almacenar, leer, modificar y presentar información, aplicando buenas prácticas de programación.

Planteamiento del problema

Una empresa de **seguridad** lleva un registro manual en papel de las actividades diarias de su equipo: revisiones, incidentes, tareas completadas y pendientes.

El director desea pasar este sistema a un formato digital ligero usando **archivos de texto**, porque todavía no quieren bases de datos.

Te han contratado para desarrollar un módulo prototipo en PHP que permita:

1. Registrar actividades en una **bitácora diaria**.
2. Guardarlas en un **archivo de texto plano**.
3. Consultar el archivo para leer todas las actividades.
4. Agregar nuevas actividades sin borrar las anteriores.
5. Mostrar la bitácora en una lista ordenada.

Actividad

Desarrollar un script en PHP que cumpla **todos** los siguientes puntos:

1. Tener un formulario HTML con:
 - **Descripción** de la actividad (texto)
 - **Responsable** (texto) ○ **Fecha** (input date)
2. Al enviar el formulario:
 - Crear (si no existe) o **abrir en modo append** un archivo llamado bitacora.txt.
 - Guardar la actividad con este formato:

Fecha: YYYY-MM-DD

Actividad: texto

Responsable: texto

3. Crear un segundo bloque de código PHP que:
 - Abra el archivo bitacora.txt.
 - Lea todo su contenido.
 - Lo muestre dentro de un `<pre>` o en una lista ordenada `<ol>`
4. Validar que no se agreguen campos vacíos.
5. Mostrar mensajes de error y éxito.

Estructura de archivos del problema:

bitacora/ index.php
bitacora.txt (se creará solo)



Ilustración 1. Screenshot de la estructura de archivos del problema.

Preguntas de reflexión y conclusiones

1. ¿Qué ventajas ofrece escribir en archivos de texto cuando no se requiere una base de datos?

Escribir en archivos de texto es más simple porque no necesitas instalar ni configurar ningún gestor de base de datos. El archivo se puede copiar o mover fácilmente entre sistemas sin dependencias externas, consume menos recursos del servidor y se implementa en pocas líneas de código. Es ideal para prototipos, registros pequeños o logs simples. La desventaja es que no escala bien: con muchos registros, buscar o filtrar información se vuelve lento e ineficiente.

2. ¿Cuál es la diferencia entre `file_put_contents` con `FILE_APPEND` sin él?
Sin `FILE_APPEND`, la función sobrescribe completamente el archivo cada vez que se llama, borrando todo el contenido anterior. Con `FILE_APPEND`, el nuevo contenido se agrega al final del archivo sin tocar lo que ya estaba guardado. Para una bitácora es obligatorio usar `FILE_APPEND`, ya que el objetivo es acumular registros. Sin él, cada nueva entrada borraría todas las anteriores.

3. ¿Cómo evitarías que usuarios maliciosos escriban código dentro del archivo?

La principal medida es usar `htmlspecialchars()` sobre todos los datos recibidos del formulario antes de guardarlos o mostrarlos. Esto convierte caracteres como `<`, `>` y `"` en entidades HTML, impidiendo que se ejecute código si el contenido se despliega en el navegador. También se puede usar `strip_tags()` para eliminar directamente cualquier etiqueta HTML o PHP del texto. Adicionalmente, conviene validar el formato de cada campo (por ejemplo, que la fecha cumpla el patrón `YYYY-MM-DD`) y nunca usar `include` o `eval` con el contenido leído del archivo.

4. ¿Qué pasa si dos usuarios escriben simultáneamente en el archivo?

Sin control, pueden ocurrir condiciones de carrera: ambos usuarios abren el archivo al mismo tiempo, escriben en paralelo y sus datos se mezclan o corrompen, dejando el archivo en un estado inconsistente. La solución es usar el flag `LOCK_EX` junto con `FILE_APPEND`. Este flag aplica un bloqueo exclusivo sobre el archivo, de modo que si un proceso está escribiendo, los demás esperan su turno antes de acceder. Para aplicaciones con mucha concurrencia, lo más recomendable sería migrar a una base de datos, que gestiona estos bloqueos de forma más robusta y automática.

5. ¿Qué diferencia habría si usaras funciones como , `fopen()` , `fwrite()` , `fclose()` ?

Ambos enfoques logran el mismo resultado, pero con distinto nivel de control. `file_put_contents()` es un atajo que hace todo en una sola línea de forma automática. En cambio, `fopen()`, `fwrite()` y `fclose()` son funciones de bajo nivel que requieren más líneas pero ofrecen mayor control: puedes elegir el modo de apertura, escribir el contenido en partes, aplicar el bloqueo de forma explícita con `flock()` y manejar errores de manera más granular. Para esta actividad, `file_put_contents()` es suficiente. Las funciones manuales serían preferibles si el archivo fuera muy grande o si necesitaras leer y escribir en el mismo archivo dentro de la misma operación.

6. Escribe tus conclusiones (¿Qué sabía? ¿Qué aprendí? ¿Qué podría hacer con lo que aprendí?)

¿Qué sabía? Sabía que PHP puede interactuar con archivos del servidor y que existen funciones para leer y escribir texto. También tenía conocimientos básicos de formularios HTML y validaciones simples con PHP.

¿Qué aprendí? Aprendí a usar `file_put_contents()` con `FILE_APPEND` para acumular registros sin perder los anteriores, y la importancia del flag `LOCK_EX` para evitar corrupción en accesos simultáneos. También reforcé el uso de `htmlspecialchars()` para proteger la aplicación contra inyección de código, y practiqué separar la lógica del script en dos bloques claros: uno para escribir y otro para leer el archivo.

Criterio	Descripción	Ponderación
Estructura correcta del formulario HTML	Captura correcta de datos	10%
Escritura correcta en archivo de texto	Guarda sin borrar contenido	25%
Lectura y despliegue del archivo	Muestra contenido de forma clara	25%
Validaciones y manejo de errores	Campos vacíos, mensajes claros	20%
Reflexión y conclusiones	Respuestas claras y coherentes	20%

JESÚS SALAS MARÍN

2